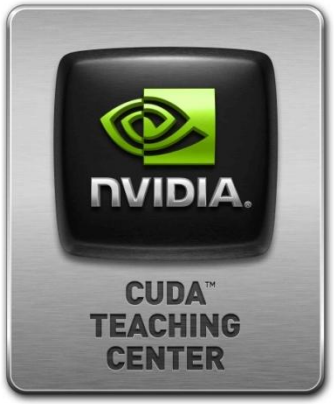
Programación masivamente paralela en procesadores gráficos (GPUs)

E. Dufrechou, P. Ezzatti, M. Freire y M. Pedemonte



Clase 7
Programación CUDA III

Contenido

- **Paralelismo dinámico**
- **Warp shuffle**
- **Planificación independiente de hilos**
- **Warp shuffle revisitado**
- **Thread Block Clusters**

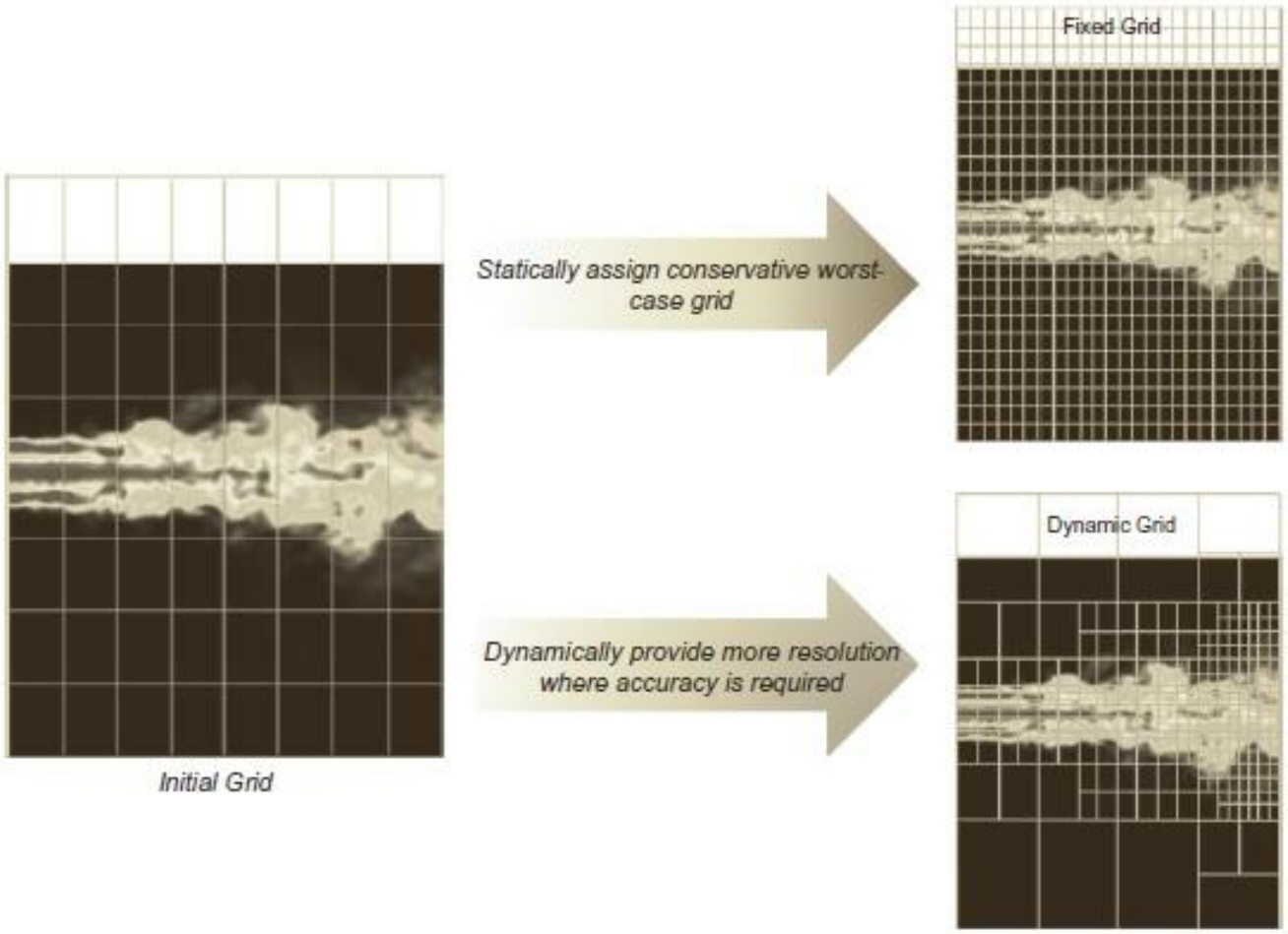
Paralelismo Dinámico

Paralelismo Dinámico

- El soporte para paralelismo dinámico se incorpora en la arquitectura Kepler.
- Antes de Kepler, los kernels solamente podían ser lanzado desde el host.
- El paralelismo dinámico permite que un kernel pueda lanzar otro kernel, creando una nueva grilla de hilos.
- Es ideal para problemas con:
  - Recursión
  - Loops irregulares

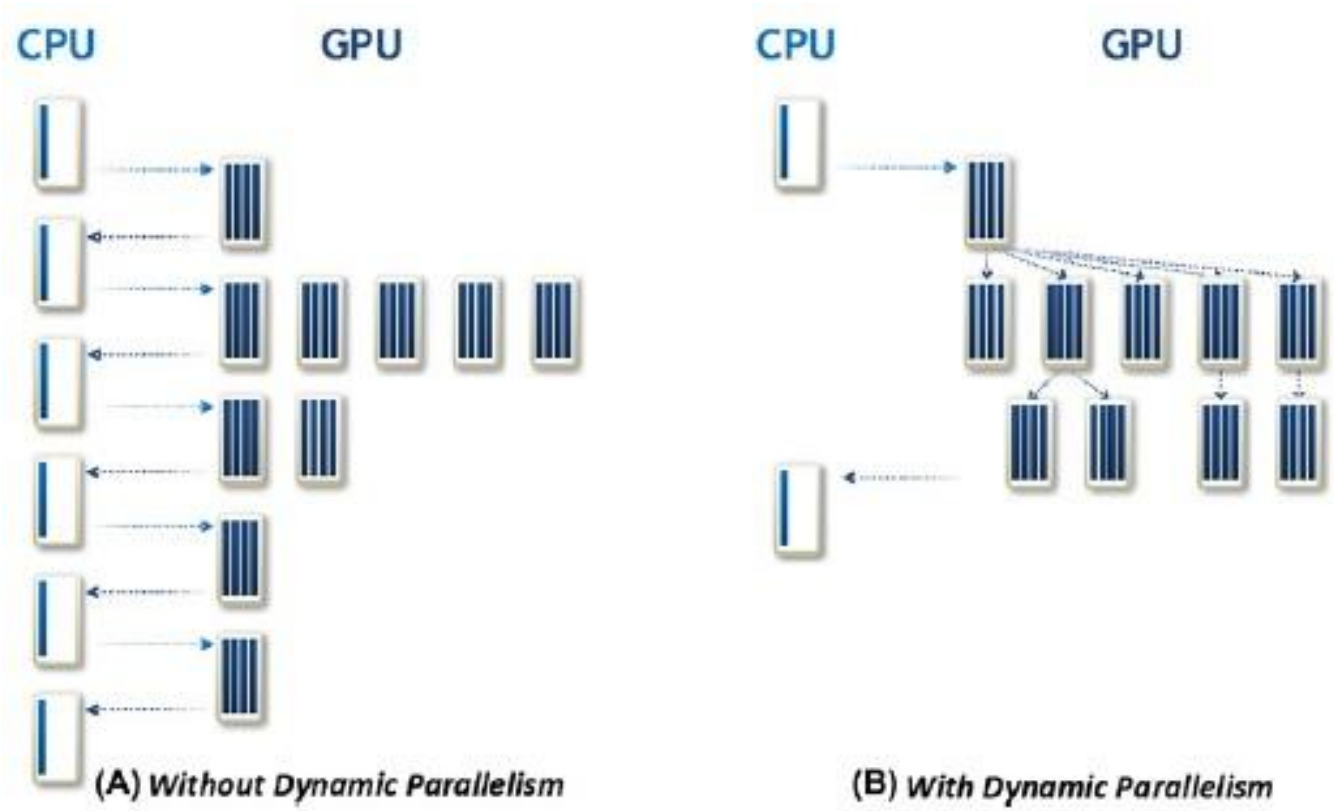
Paralelismo Dinámico

- **El paralelismo dinámico permite que el algoritmo sea capaz de descubrir dinámicamente en dónde concentrar el trabajo.**



Paralelismo Dinámico

- **Sin paralelismo dinámico también se pueden lograr los refinamientos pero es engorroso.**
- **Todo debe ser comunicado al host para que refine.**

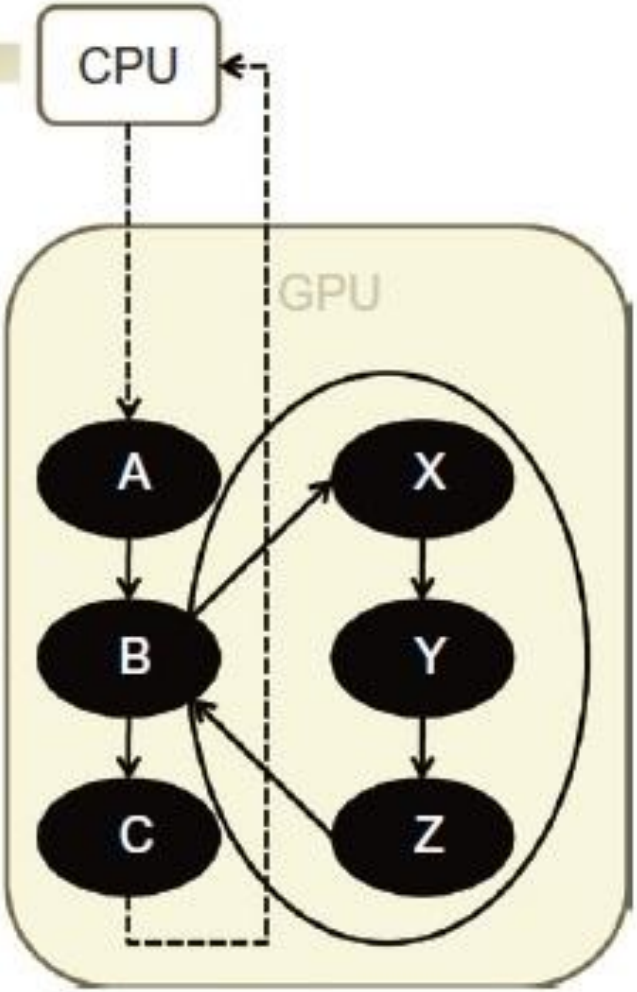


Paralelismo Dinámico

- **La invocación es igual que en el caso de un kernel convencional.**

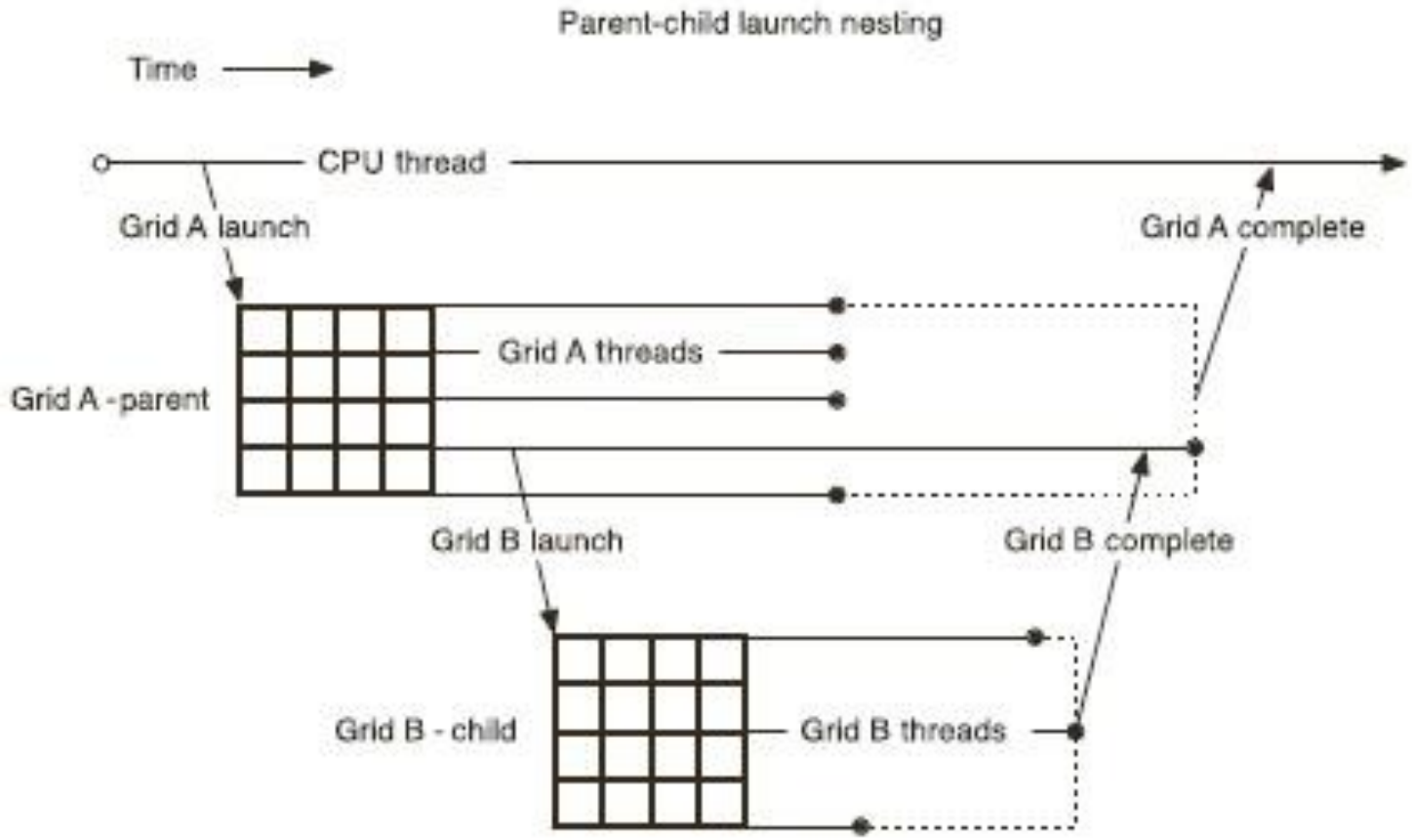
```
int main() {
    float *data;
    setup(data);

    A <<< ... >>> (data);
    B <<< ... >>> (data);
    C <<< ... >>> (data);

    cudaDeviceSynchronize();
    return 0;
}
```

```
__global__ void B(float *data)
{
    do_stuff(data);

    X <<< ... >>> (data);
    Y <<< ... >>> (data);
    Z <<< ... >>> (data);
    cudaDeviceSynchronize();

    do_more_stuff(data);
}
```



Paralelismo Dinámico

- **Un hilo puede hacer la llamada.**
- **El tiempo de vida es lo que uno esperaría.**



Paralelismo Dinámico

- **Existe coherencia en memoria global entre las grillas padres e hijos.**
- **No se puede compartir la memoria local (incluidos los registros) entre los padres e hijos.**
- **Tampoco se puede pasar la memoria compartida de padres a hijos.**
- **No se explica exactamente la ejecución pero se deduce que suspende la ejecución del padre. (en qué multiprocesador?)**

El paralelismo dinámico no es muy usado en la práctica, pero puede servir para algunas aplicaciones específicas.

Warp Shuffle

Warp Shuffle

- El soporte para operaciones del tipo warp shuffle se incorpora en la arquitectura Kepler.
- En muchos casos de programas en CUDA se necesita comunicar valores entre los hilos de un bloque.
- Por ejemplo, en el caso de las reducciones.
- La forma tradicional de hacer esa comunicación es a través del uso de shared memory.
- Desde la arquitectura Kepler, los hilos de un warp pueden leer los datos de los registros de otros hilos del warp.
- Para eso se usa las instrucciones shuffle (SHFL).

Warp Shuffle

- **Las operaciones shuffle son más rápidas que usar shared memory en forma segura (es decir incluyendo las sincronizaciones).**
- **Shuffle solamente involucra una operación, mientras que en shared se requieren tres operaciones: escritura, sincronización y lectura.**
- **Las operaciones shuffle no son más lentas que usar shared memory en forma insegura (es decir asumiendo que hay operaciones que se dan al mismo tiempo y omitiendo las sincronizaciones).**
- **Shuffle no requiere las sincronizaciones porque está implícito que el warp ejecuta todo en el mismo ciclo (veremos más adelante que eso ha cambiado últimamente).**

Warp Shuffle

- **Las operaciones shuffle son:**
  - `int __shfl(int var, int srcLane, int width=32)`
  - `int __shfl_up(int var, unsigned int delta, int width=32)`
  - `int __shfl_down(int var, unsigned int delta, int width=32)`
  - `int __shfl_xor(int var, int laneMask, int width=32)`
- **El parámetro width**
  - **es por defecto 32**
  - **tiene que ser una potencia de 2 entre 2 y 32**
  - **permite subdividir el warp en subsecciones (si width < 32)**
  - **Cada subsección se considera por separado y el intercambio solo puede ser entre hilos de una misma subsección.**

Warp Shuffle

- Se usa el nombre carriles (lanes) para referirse a los hilos de un warp.
- Los carriles pueden tener como índice entre 0 y warpSize-1 (inclusive).

- __shfl(var, srcLane):
  - devuelve el valor de la variable var almacenado por el hilo cuyo carril es srcLane.
  - Esta operación sirve para hacer broadcast de valores dentro de un warp.

Warp Shuffle

- __shfl_xor(var, laneMask):
  - devuelve el valor de la variable var almacenado por el hilo cuyo carril se calcula a partir de un XOR bit a bit (bitwise) entre el carril y laneMask.

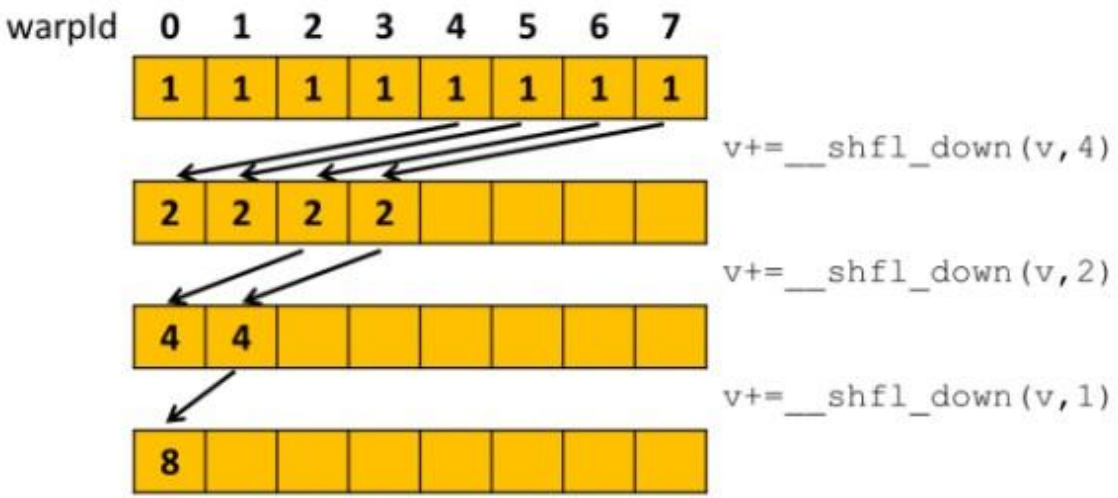
- __shfl_down(var, delta)
  - Calcula el índice del carril sumando delta al índice del que invoca y sujeto al rango (solo se activan los que dan un índice dentro del rango).
  - Sirve para copiar valores de un carril con un índice más alto que el que invoca.

Warp Shuffle

- __shfl_up(var, delta)
  - Calcula el índice del carril restando delta al índice del que invoca y sujeto al rango (solo se activan los que dan un índice dentro del rango).
  - Sirve para copiar valores de un carril con un índice más bajo que el que invoca.

Warp Shuffle

- **Ejemplo: Reducción de un warp**




```
__inline__ __device__
int warpReduceSum(int val) {
  for (int offset = warpSize/2; offset > 0; offset /= 2)
    val += __shfl_down(val, offset);
  return val;
}
```

Warp Shuffle

- **Ejemplo: Reducción de bloque**

```
__inline__ __device__
int blockReduceSum(int val) {

  static __shared__ int shared[32]; // Shared mem for 32 partial sums
  int lane = threadIdx.x % warpSize;
  int wid = threadIdx.x / warpSize;

  val = warpReduceSum(val);     // Each warp performs partial reduction

  if (lane==0) shared[wid]=val; // Write reduced value to shared memory

  __syncthreads();              // Wait for all partial reductions

  //read from shared memory only if that warp existed
  val = (threadIdx.x < blockDim.x / warpSize) ? shared[lane] : 0;

  if (wid==0) val = warpReduceSum(val); //Final reduce within first warp

  return val;
}
```

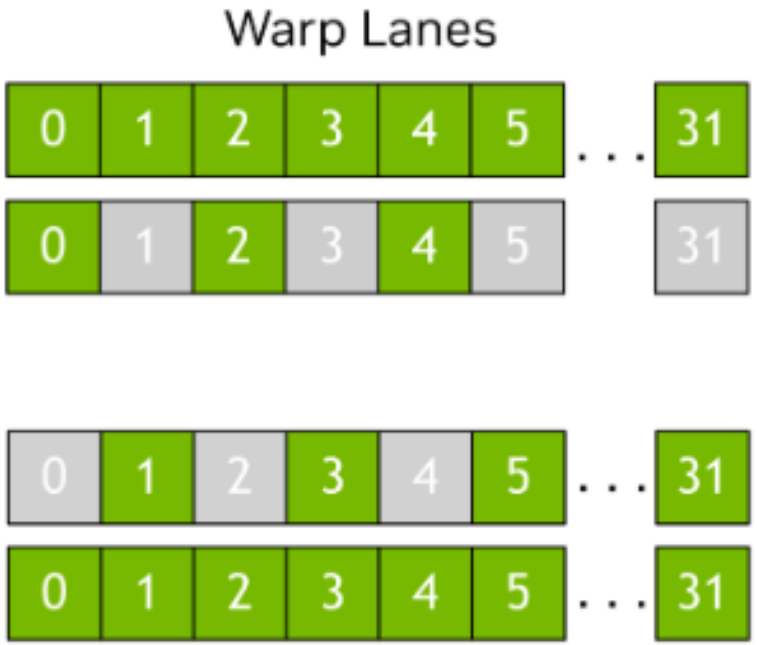
Warp Shuffle

- **Ventajas de las operaciones warp shuffle:**
  - **Son más rápidas que la memoria shared.**
  - **Reducen la cantidad de shared memory necesaria, por lo que permiten usar más.**
  - **En las distintas arquitecturas, a veces aumenta más la cantidad de cores que la shared disponible.**

Planificación independiente de hilos

Planificación independiente de hilos

```
if(threadIdx.x%2 == 0)
   {a = r(t); }
else
   {a = q(t); }
y = f(a);
```

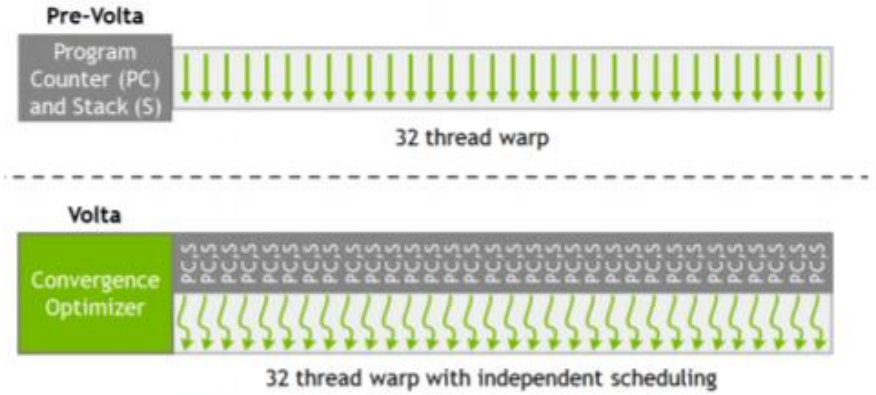


Planificación independiente de hilos

- **En las GPU con capacidad de cómputo inferior a 7.0, los warps utilizaban un único contador de programa compartido por los 32 hilos del warp y una máscara de activación que especificaba los hilos activos del warp.**
- **Por este motivo, los hilos de un warp en regiones divergentes o en diferentes estados de ejecución no pueden sincronizarse ni intercambiar datos.**

Planificación independiente de hilos

- **Desde el punto de vista de las capacidades de programación, la modificación más importante es la planificación independiente de los hilos.**
- **Hasta Pascal el modelo de ejecución es SIMT con divergencia y la suspensión de hilos.**
- **Cada hilo independiente mantiene su estado de ejecución, incluyendo un program counter y el stack de llamadas.**



Volta (bottom) independent thread scheduling architecture block diagram compared to Pascal and earlier architectures (top). Volta maintains per-thread scheduling resources such as program counter (PC) and call stack (S), while earlier architectures maintained these resources per warp.

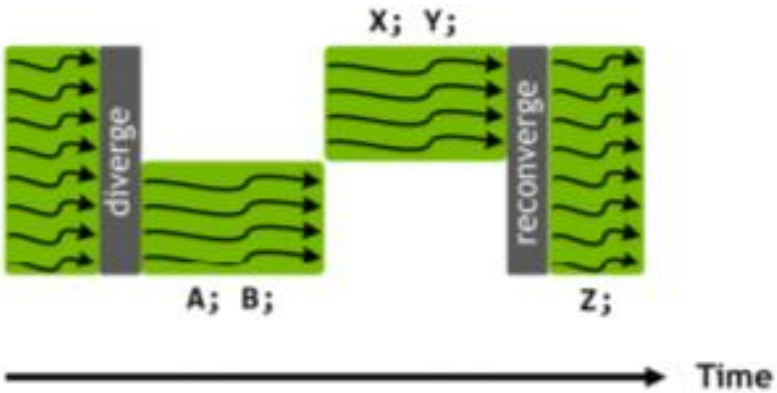
Planificación independiente de hilos

- **A partir de la arquitectura Volta incluye un optimizador de la planificación que determina cómo agrupar los hilos activos del mismo warp en las unidades SIMT.**
- **Esto conserva el alto rendimiento de ejecución SIMT pero con mucha más flexibilidad: los hilos pueden diverger y reconverger a nivel de subwarp.**
- **El optimizador se encarga de agrupar los hilos que ejecutan el mismo código y los ejecuta en paralelo para lograr la máxima eficiencia.**

Planificación independiente de hilos

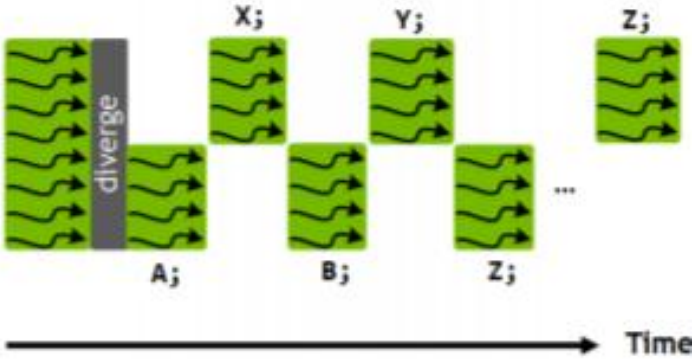
SIMT Warp Execution Model of Pascal and Earlier GPUs

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



Volta Independent Thread Scheduling

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



Planificación independiente de hilos

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```

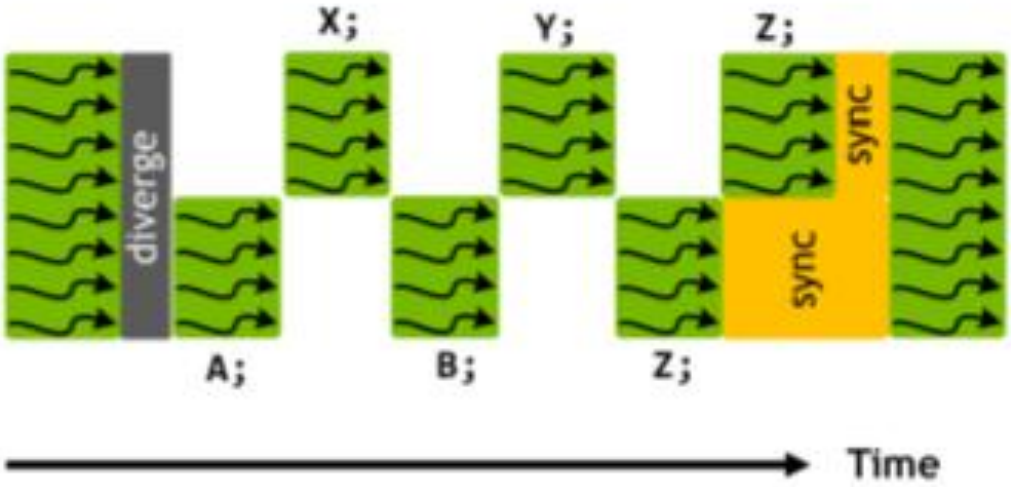


Figure 23. Programs use Explicit Synchronization to Reconverge Threads in a Warp

Planificación independiente de hilos

- La planificación independiente de hilos puede romper código que depende del comportamiento implícitamente síncrono a nivel de warp de arquitecturas anteriores.
- El código síncrono a nivel de warp asume que los hilos dentro de un mismo warp se ejecutan a la misma vez en cada instrucción
- La capacidad de los hilos para divergir y reconverger con granularidad inferior a la de un warp hace que eso deje de ser válido.
- Se debe sincronizar explícitamente utilizando __syncwarp() para garantizar un comportamiento correcto (por ejemplo, en las reducciones intra-warp sin sincronización).

Warp Shuffle Revisitado

Warp Shuffle Revisitado

- **Las operaciones shuffle agregan sync al final y una máscara para indicar que hilos del warp deben estar activos.**
- **Las operaciones son**
  - `int __shfl_sync(unsigned mask, int var, int srcLane, int width=32)`
  - `int __shfl_up_sync(unsigned mask, int var, unsigned int delta, int width=32)`
  - `int __shfl_down_sync(unsigned mask, int var, unsigned int delta, int width=32)`
  - `int __shfl_xor_sync (unsigned mask, int var, int laneMask, int width=32)`

Warp Shuffle Revisitado

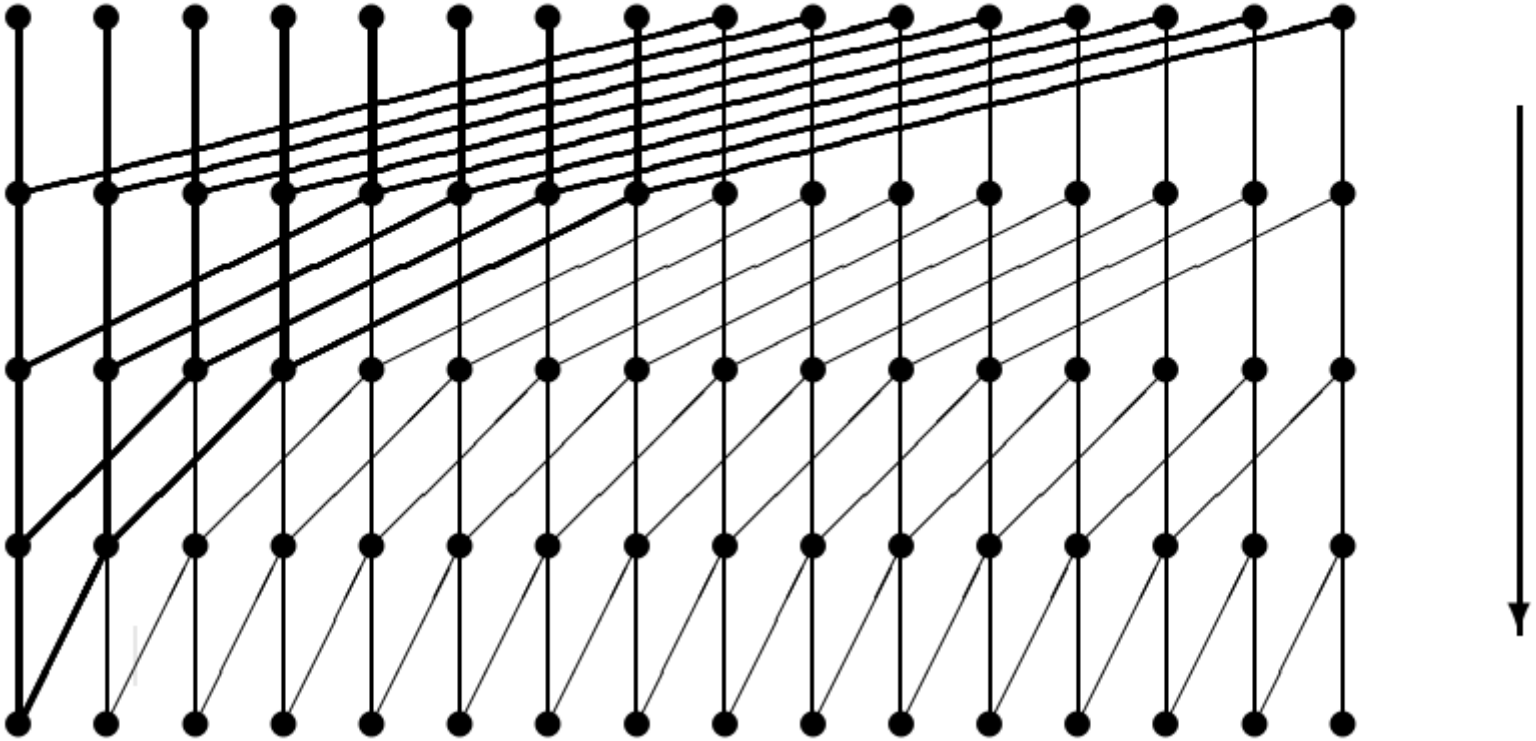
- **El parámetro mask**
  - **indica qué hilos del warp participan en la llamada.**
  - **cada bit de mask corresponde al lane ID de un hilo**
  - **la función espera hasta que todos los hilos del warp no finalizados especificados en mask alcancen la llamada.**
  - **En general se setea en 0xffffffff o equivalentemente en -1 (es decir todo en 1).**
- **Para una ejecución correcta deben cumplirse las siguientes restricciones:**
  - **Cada hilo que realiza la llamada debe tener su bit correspondiente activado en la mask.**
  - **Cada hilo que no realiza la llamada debe tener su bit correspondiente en cero en la mask.**

Warp Shuffle Revisitado

- **Los hilos solo pueden leer datos de otro hilo que esté participando activamente en la operación de shuffle.**
- **Si el hilo está inactivo, el valor obtenido es indefinido.**
- **Hay que tener cuidado con el código condicional o usar Cooperative Groups.**
- **También se agrega una operación de sincronización del warp: __syncwarp().**

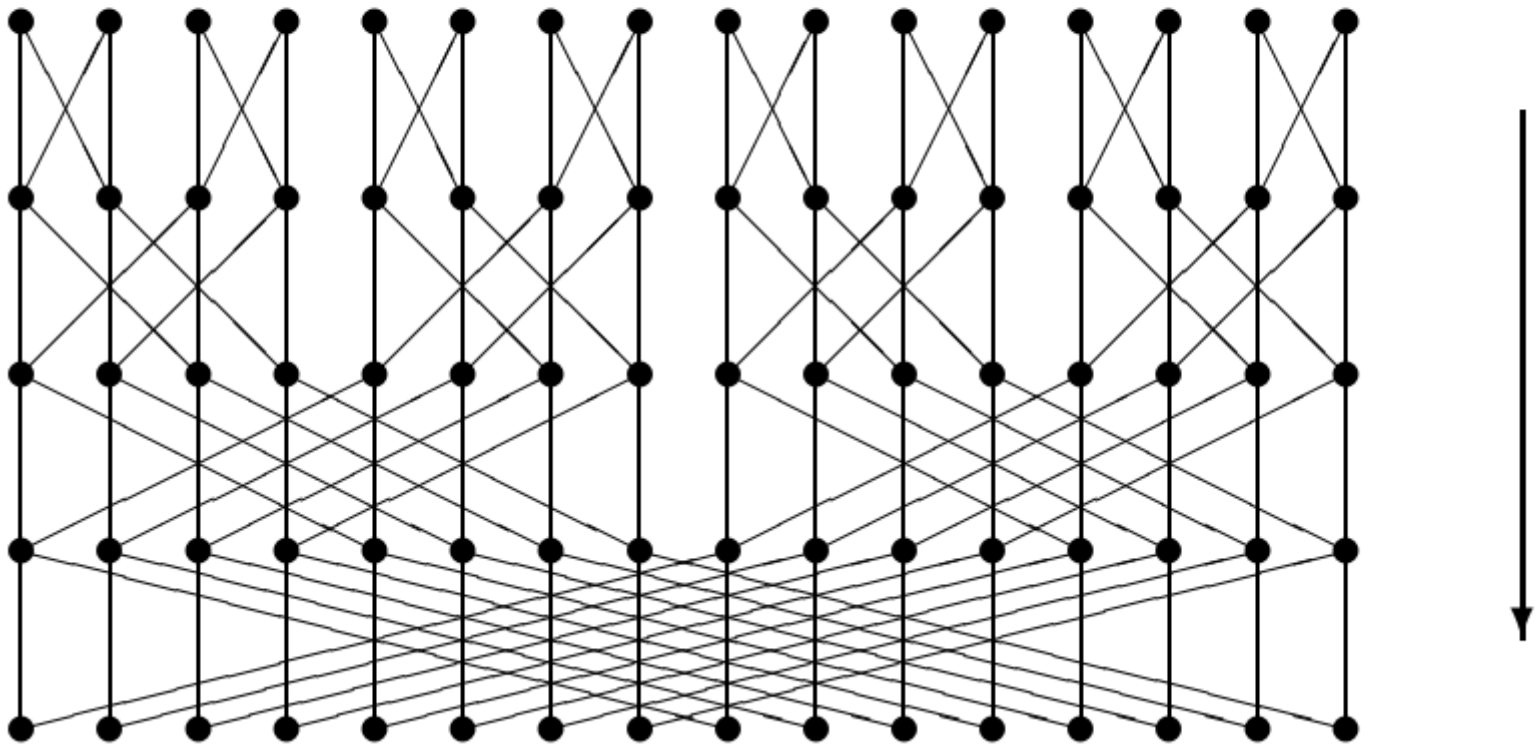
Warp Shuffle Revisitado

```
for (int i=warpSize/2; i>0; i=i/2)
  value += __shfl_down_sync(-1, value, i);
```



Warp Shuffle Revisitado

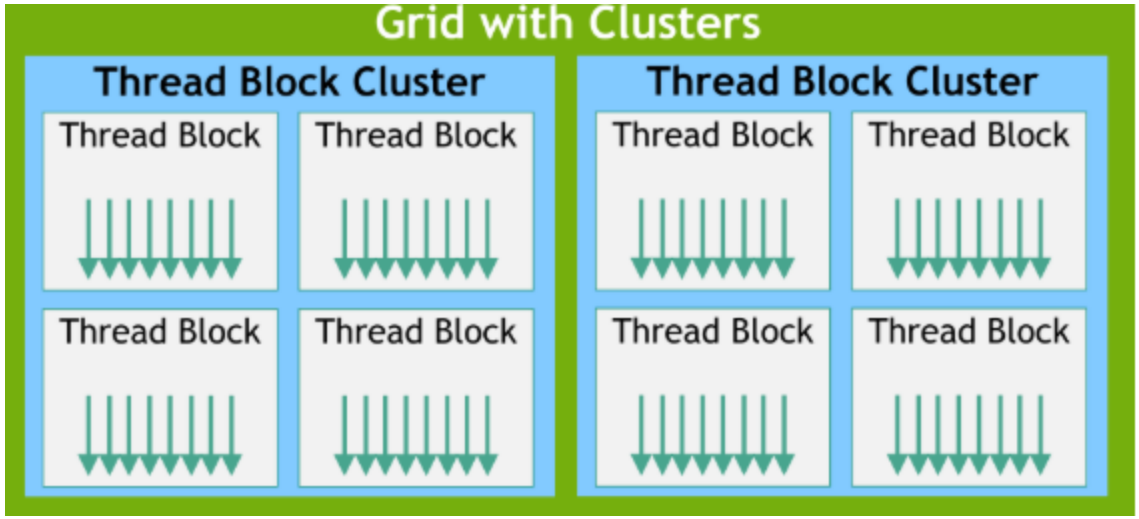
```
for (int i=1; i<warpSize; i*=2)
  value += __shfl_xor_sync(-1, value, i);
```



Thread Block Clusters

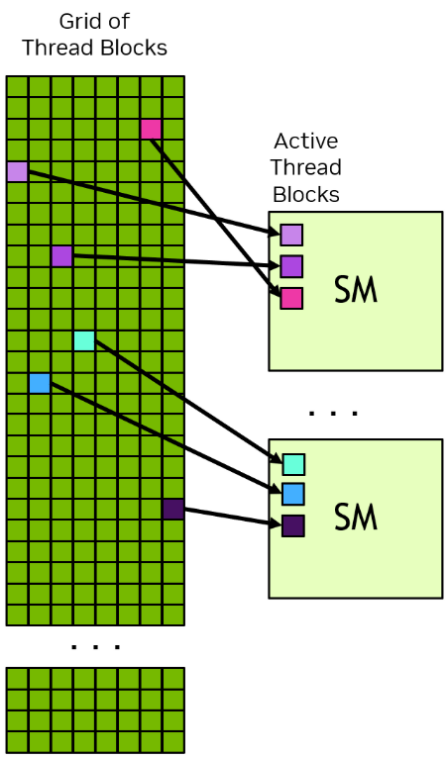
Thread Block Clusters

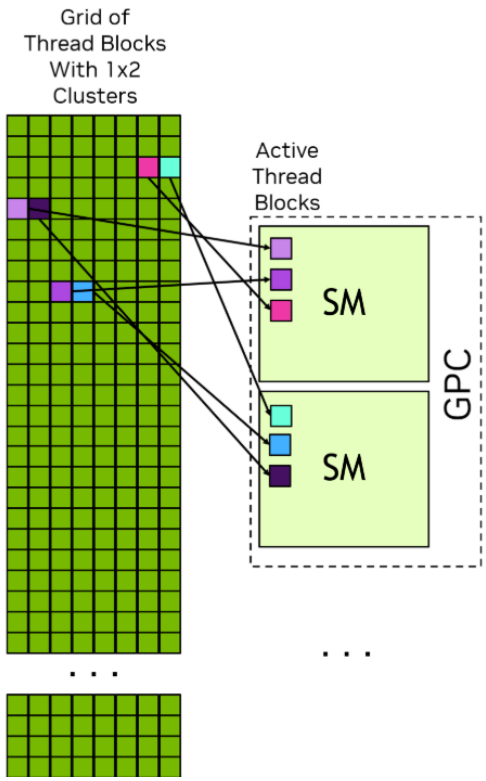
- Agregados a partir de CUDA 9
- Los clústeres son un grupo de bloques de hilos.
- Pueden organizarse en 1, 2 o 3 dimensiones.
- Especificar clústeres no cambia las dimensiones de la grilla, ni los índices de un bloque de hilos dentro de la grilla.



Thread Block Clusters

- **Proporciona oportunidades para la sincronización y la comunicación a nivel de clúster.**
- **Se garantiza que un clúster ejecuta en una misma GPC**





Thread Block Clusters

- **Los hilos que pertenecen a bloques distintos, pero dentro del mismo clúster, pueden comunicarse y sincronizarse entre sí mediante interfaces de software proporcionadas por Cooperative Groups.**
- **Cooperative Groups es a nivel de software (es decir no es a nivel del modelo de programación CUDA)**
  - **permite definir grupos de hilos que pueden sincronizarse entre sí, incluso si abarcan múltiples bloques de hilos, múltiples grillas en una sola GPU, o incluso varias GPU.**
  - **Usarlos implica limitaciones semánticas y de rendimiento**
  - **No entraremos en mayores detalles hoy.**

Thread Block Clusters

- **Los hilos en un mismo clúster pueden acceder a la memoria compartida de todos los bloques del clúster.**
- **A esto se denomina memoria compartida distribuida.**
- **Los hilos que pertenecen a un TBC pueden leer, escribir o realizar operaciones atómicas en el espacio de direcciones distribuido, independientemente de si la dirección pertenece al bloque de hilos local o a un bloque de hilos remoto.**
- **Las especificaciones del tamaño de la memoria compartida, ya sea estática o dinámica, siguen siendo por bloque de hilos.**

Thread Block Clusters

- **Agrega nuevos problemas:**
  - **El acceso a datos en la memoria compartida distribuida requiere que todos los bloques de hilos existan**
  - **También es necesario garantizar que todas las operaciones sobre la memoria compartida distribuida se realicen antes de que finalice un bloque de hilos**
  - **Si un bloque de hilos remoto intenta leer la memoria compartida de un bloque de hilos dado, el programa debe garantizar que la lectura de la memoria compartida por parte del bloque remoto se complete antes de que dicho bloque pueda finalizar.**
  - **cluster.sync()**
- **No entraremos en más detalles, los interesados pueden revisar la CUDA Programming Guide.**